electronics

MDPI

Article

MoRe: LLM-Based Domain Model Generation with Hybrid Self-Refinement

Ru Chen, Jingwei Shen and Xiao He *

School of Computer and Communication Engineering, University of Science and Technology Beijing, No. 30, Xueyuan Road, Haidian District, Beijing 100083, China; b2234677@ustb.edu.cn (R.C.); jingweishen2022@126.com (J.S.)

* Correspondence: hexiao@ustb.edu.cn

Abstract

Generating domain models from requirements is a vital and complex challenge in automated software engineering. Although large language models (LLMs) have exhibited significant competence in this area, their propensity for hallucination frequently results in models that are redundant, inconsistent, or structurally unsound. To enhance the quality of automatically generated models, this paper introduces MoRe, an LLM-based approach to domain model generation with self-refinement. Within our approach, an LLM is first tasked with producing an initial domain model draft. Subsequently, a hybrid refinement—combining LLMs with a rule-based scanner—is employed to identify and correct common issues in the model. An empirical study was conducted using 30 domain modeling problems and four open-source LLMs. The results indicate that MoRe significantly improves the quality of generated domain models. This paper advocates for incorporating a self-refinement phase as a standard component in any automated modeling workflow.

Keywords: domain modeling; large language model; model generation; class diagram; self-refinement; software designs

1. Introduction

Large language models (LLMs), such as GPT, Claude, Gemini, Llama, DeepSeek, and Qwen, are deep neural networks trained on massive text datasets to understand, generate, and manipulate both human and programming languages [1]. LLMs have demonstrated remarkable capabilities in many software development tasks [2], such as code generation [3–6], code summarization [7,8], comment generation [9], test case generation [10,11], and bug fixing [12,13]. While LLMs excel at solving code-related tasks, their application to higher-level software design tasks—particularly domain modeling—remains relatively underexplored in the literature. This oversight is significant given modeling's critical role in software development and maintenance.

In object-oriented methodology, domain modeling is a process of translating a business requirement into a class diagram that reflects the core concepts and relationships existing in the business domain. The process fundamentally differs from coding—requiring conceptual abstraction, contextual comprehension, and semantic pattern identification that extends beyond syntactic manipulation. Existing research has primarily focused on guiding LLMs to construct domain models through two main approaches: (1) improved prompting techniques—such as in-context learning and chain-of-thought reasoning [14,15]—and

Check for updates

Academic Editor: Arkaitz Zubiaga

Received: 12 February 2026
Revised: 8 March 2026
Accepted: 10 March 2026
Published: 17 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

(2) decomposing the modeling process into subtasks [16,17], such as entity identification and relationship extraction, rather than generating the entire model in a single step.

In this paper, we propose a new perspective on LLM-based domain modeling: *can we enhance the quality of generated models through an automated process of review and revision?* While LLMs can now handle complex tasks via simple prompts due to their rapid advancement, they are inevitably plagued by the persistent issue of hallucination [18]. Based on this observation, this paper proposes MoRe, an LLM-based approach to domain model generation with self-refinement. First, MoRe generates a draft domain model by prompting an LLM with a simple instruction and a textual domain description. This draft is then fed into a three-step self-refinement post-processing pipeline, where it undergoes sequential enhancement and correction. Specifically, the three-step procedure consists of (1) model correction, (2) model reduction, and (3) rule-based model fixing. Note that the general idea of self-refinement was proposed by Madaan et al. [19] and has been widely adopted in many LLM-based SE tasks. This work applies this idea to domain modeling and proposes a new hybrid pipeline that combines both LLMs and rule-based checkers.

For evaluation, we first conducted an experiment using 30 domain modeling problems across four open-source LLMs: DeepSeek-V3.2, DeepSeek-R1-Distill-Llama-8B, Qwen3-Coder-480B-A35B-Instruct, and Qwen3-Coder-30B-A3B-Instruct. This experiment compared MoRe against two baseline approaches. The results demonstrate that MoRe improves the quality of generated domain models, as measured by F1 scores, over the baselines for most of the LLMs tested. Additionally, we performed an ablation study to assess the contribution of each step in the self-refinement pipeline. The findings confirm that the complete three-step procedure achieves the most consistent and balanced performance across different evaluation metrics and LLMs.

The contributions of this paper are summarized as follows:

- We propose MoRe, an LLM-based approach to domain model generation with a hybrid self-refinement pipeline.
- We construct a benchmark for domain model generation that contains 30 domain requirements along with reference models. Additionally, we implement a semantic model matcher to assess how close a generated model and a reference model are.
- We conduct an empirical evaluation and demonstrate the effectiveness of MoRe.

The remainder of this paper is organized as follows: Section 2 covers background and related work. Section 3 details the MoRe system, while Section 4 describes its semantic matching algorithm. Section 5 outlines the experimental design, and Section 6 presents the results. Section 7 addresses the research questions, and Section 8 examines the work's strengths and limitations. Finally, the last section concludes and suggests future work.

2. Background and Related Work

2.1. Domain Modeling

Domain modeling is a process of creating an abstract representation (i.e., a domain model) of a business domain, which focuses on its core concepts and relationships [20–22], based on a textual domain requirement. From the perspective of software architecture description [23], this process corresponds to defining the static structural dimension of the Logical View. Typically, a domain model is defined as a class diagram using object-oriented (OO) modeling concepts such as classes, attributes, references (i.e., associations and compositions), inheritance, as well as enumerations and their literals. Attributes and references are also called *structural features* of classes. This paper adheres to the fundamental OO concepts common in prior research, omitting advanced elements like stereotypes, association classes, interfaces, and n-ary relationships for consistency.

Figure 1 shows a simple example of domain modeling. A modeler is given a domain requirement that describes a business domain. Their task is to create a domain model (i.e., a class diagram) that defines the core concepts and relationships based on the requirement. A domain model can be encoded into certain modeling languages, such as PlantUML, Ecore, and UML. Figure 1 illustrates a PlantUML model created from the requirement. The model consists of four classes (including `Company`, `Department`, `Office`, and `Employee`), three compositions (denoted as `*--`) and two associations (denoted as `-->`). This PlantUML model may further be visualized (see the visualized domain model) or converted into other formats, e.g., Ecore.

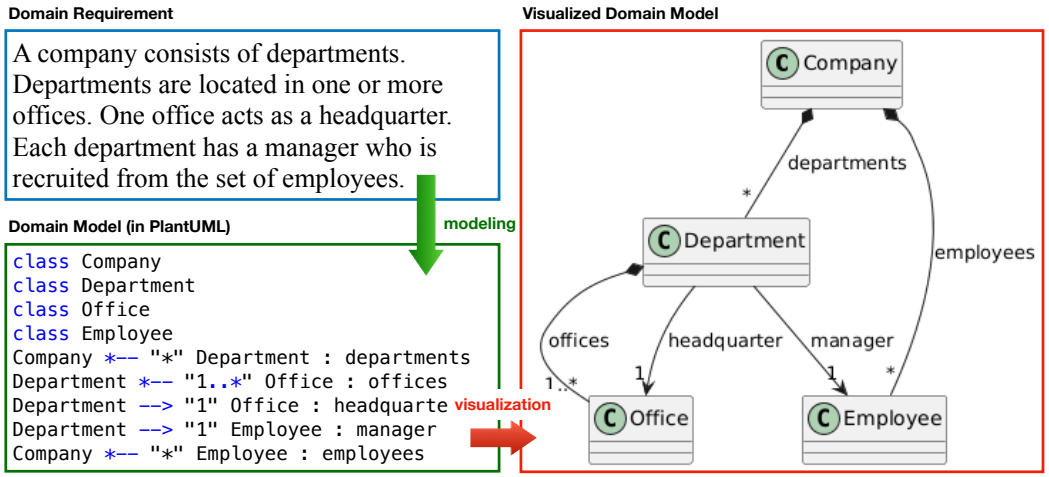


Figure 1. Example of domain modeling.

2.2. Related Work

Automated modeling has attracted considerable interest in the model-driven engineering (MDE) and software engineering communities.

Conventional solutions for deriving models from natural language (NL) descriptions often combine Natural Language Processing (NLP) with rule-based strategies [24,25]. For instance, Tao Yue et al. [26] proposed Restricted Use Case Modeling (RUCM), which applies specific restriction rules and a modified template to textual use case specifications to reduce NL ambiguity. The structured RUCM specifications are then parsed using NLP to build sentence structures, which serve as input for automated model transformations, generating analysis models such as class and sequence diagrams. Similarly, Thakur et al. [27] present an NLP-based approach for domain model generation. Their method begins with NLP analysis to extract sentence structures from textual descriptions, followed by the application of defined analysis rules—each designed to identify a specific type of model element (e.g., classes, attributes, and inheritance relationships) from these structures. Applying these rules to the parsed sentences enables the automatic construction of a class diagram. Other related work includes approaches by Lucassen et al. [28] and Arora et al. [29], which also rely on heuristic rules to generate models from NL descriptions, further illustrating the rule-driven paradigm in this research area. The common limitation of these rule-based NLP approaches is their inherent lack of flexibility and scalability due to dependence on manually crafted, domain-specific transformation rules.

With the enhanced capabilities of large language models (LLMs) in processing both natural language (NL) and source code, utilizing LLMs as an alternative to traditional NLP techniques for model generation has become feasible. Camara et al. [30] documented their experience using ChatGPT 3.5 for model generation, highlighting several pitfalls such as grammatical errors, incorrect semantic interpretations, inconsistent responses, and issues with scalability. Chen et al. [31] investigated the impact of various prompt

engineering strategies (zero-shot, few-shot, and chain-of-thought) on domain modeling, and evaluated the quality of domain models via human assessment. Yang et al. [32] proposes an iterative approach that decomposes the domain modeling process into four LLM-driven substeps: identifying classes and attributes, identifying the player-role pattern, self-reflection, and inferring relationships, where the output of each stage serves as the input to the next. The self-reflection step serves a similar function as our self-refinement pipeline, but focuses exclusively on classes and attributes. Silva et al. [33] adopt a multi-step generation strategy similar to that of Yang. However, they enhance the approach by prompting the LLM to return multiple candidate answers for each query. The best answer is then selected through a majority voting process at each step, which is also performed by the LLM itself. Besides UML models, there is also some work on SysML model generation [34,35].

Our work also builds upon LLMs. However, rather than guiding an LLM to iteratively construct a model from scratch, our primary focus is on refining an initial, LLM-generated model. To this end, we first identify a set of common issues in the generated models. We then employ a hybrid approach—combining a rule-based checker with targeted LLM queries—to detect and correct these issues, thereby improving the final model's quality. Our work does not conflict with existing efforts; rather, it complements them.

It is also crucial to contextualize the output of our approach within the broader framework of software architecture. As analyzed in recent studies on software architecture description [23], a comprehensive architectural representation typically follows the "1 + 5" view model. Within this framework, our work explicitly targets the Logical View, specifically focusing on its static structural dimension (i.e., classes, attributes, and relationships). We acknowledge that a complete logical view in detailed software design necessitates the identification of methods (operations) to define object behaviors [23]. However, our work focuses on the domain modeling phase—a conceptualization stage aimed at establishing a precise domain vocabulary and structural foundation (i.e., information coverage)—rather than the detailed specification of software operations. Consequently, our approach effectively establishes the structural backbone of the logical view, leaving the generation of behavioral specifications as a distinct scope for future integration.

3. The MoRe Approach

This section proposes MoRe, an LLM-based approach to domain model generation with hybrid self-refinement. The overall workflow of MoRe is illustrated in Figure 2 by example. The process of converting a textual requirement into a domain model consists of four major steps. Step ① is *initial generation*, in which an LLM is prompted to produce an initial domain model based on the given domain requirement. Step ②, *model correction*, follows: the LLM is asked to detect specific semantic issues in the initial model and correct them. In step ③, *model reduction*, the corrected model is sent again to the LLM to identify and remove redundant fragments. Finally, step ④, *rule-based fixing*, employs a rule-based checker to scan the current model. This produces two categories of issues: *confirmed issues with known fixes* and *pending issues*. For the first category, MoRe applies algorithmic fixes using predefined strategies. For the second category, MoRe queries the LLM to decide how to resolve them. After addressing these issues, MoRe outputs the *final domain model*. The prompt templates used in MoRe are presented in Appendix A.

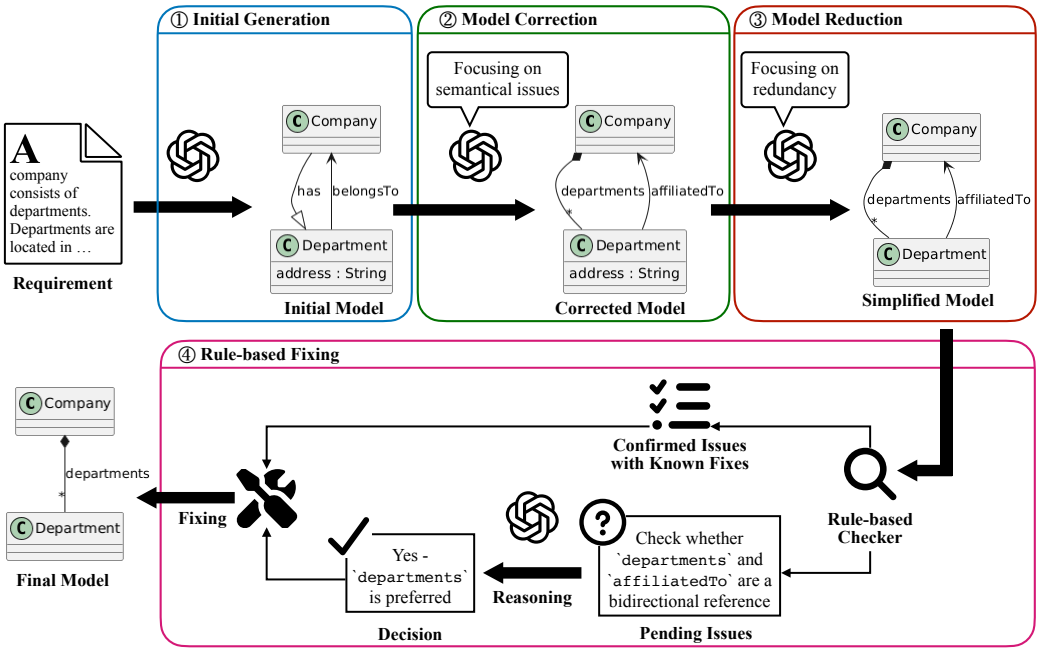


Figure 2. Illustrative example of MoRe workflow.

3.1. Initial Generation

The first step of MoRe is to generate a draft domain model from the given requirement using an LLM. Since MoRe focuses primarily on the refinement phase, we use only a simple prompt to guide the model generation. The prompt begins by defining the LLM's role and task: "You are a domain modeling expert for UML class diagrams. You must carefully analyze <Description> and create a domain model that represents the domain knowledge provided." It then provides the domain description and specifies the expected output format. To facilitate comprehension and generation by LLMs, we consistently use PlantUML as the encoding format for domain models when interacting with the LLM.

Upon receiving this prompt, the LLM returns an initial domain model draft. This draft, however, is likely to contain various semantic and syntactic flaws. MoRe is designed to refine the model and minimize such flaws, thereby enhancing its overall quality. To understand what flaws LLMs tend to produce, we used the same simple prompt to interact with DeepSeek V3.2 and Qwen3-Coder-480B-A35B-Instruct and collected 60 samples. By analyzing these samples, we identified a set of recurring issues and quantified their frequency, which are summarized in Table 1. The frequency was calculated per sample, counting each issue only once per sample regardless of how many times it occurred within that sample. We group these issues into three types: *semantics missing*, *conceptual error*, and *redundancy*. Based on these common issues, MoRe proceeds to the refinement phase.

- *Semantics missing* means that the generated model is incomplete, resulting in the absence of critical semantic elements. Table 1 lists two issues of semantics missing that are related to reference names, as these can be effectively addressed. While a generated model may also exhibit other issues of this type—such as missing implied classes—relying on LLMs to analyze them could introduce many false positives and thereby degrade model quality.
- *Conceptual error* refers to cases where an LLM incorrectly applies modeling concepts while constructing a domain model. Table 1 outlines four such issues. For instance, *misuse of class as primitive type* describes the inclusion of a class that semantically corresponds to a primitive type.
- *Redundancy* refers to situations in which the generated model contains over-designed fragments that can be removed without affecting its meaning. For example, if a class

in the model has no attributes and participates in no relationships—as in the case of an *unnecessary class*—it can be considered redundant and may be removed.

Table 1. Common issues in initially generated models.

Name	Type	Symptom	Freq.
Using generic name	SM	A reference name is generic, such as `has`, `contains`, and `refersTo`, and fails to provide meaningful information. e.g., in the initial model in Figure 2, `Company` and `Department` are linked via `has`.	13.6%
Empty name	SM	Some references may not have names.	5.7%
Misuse of class as primitive type	CE	A class is defined to denote a primitive type or an enumeration. e.g., a class named `Demical`.	6.8%
Misuse of association as inheritance	CE	An association is defined to denote an inheritance between classes. e.g., an association with a name `is_a` or `inherits`.	1.1%
Misuse of association as message passing	CE	An association is defined to denote a message passing or a procedure call between classes.	13.6%
Misuse of inheritance as reference	CE	An inheritance relationship is defined to denote a reference. e.g., in the initial model in Figure 2, the relationship `has` must be a composition (`*--`) rather than an inheritance (`--|>`).	18.2%
Unnecessary class	R	The model includes a class that is not referenced in the description yet holds no structural data and exhibits no meaningful behavior.	6.8%
Unnecessary structural features	R	The model includes an attribute/reference that can be computed by other structural features. For example, the class `Person` has both `firstName` and `lastName`, so `fullName` is an unnecessary attribute.	34.1%

SM = Semantics Missing; CE = Conceptual Error; R = Redundancy. Freq. = Frequency in the LLMs samples.

3.2. Model Correction

The second step of MoRe is *model correction*, which aims to address the issues of semantics missing and conceptual error. This step is further divided into two sequential subtasks: *name correction* and *concept correction*.

- MoRe first performs *name correction* by instructing an LLM to refine the names of associations and compositions based on the domain requirement. The LLM is prompted to act as a senior modeler and to examine every reference in the model. It is given three guidelines: (1) each relationship must receive a meaningful, semantics-based name; (2) generic or verb-based labels (e.g., `has`, `contains`, `refers_to`) are prohibited; and (3) the output must adhere to a specified format that is easy for parsing. For example, as shown in Figure 2, the relationship name `has` in the initial model is changed to `departments` in the corrected model.
- Afterward, MoRe performs *concept correction* using the LLM. The LLM is instructed to review the domain model thoroughly and scan for any conceptual misuse. MoRe provides three guidelines for this task: (1) a complete list of conceptual errors that the LLM must verify; (2) the instruction for adjusting any unreasonable inheritance hierarchies, as fixing conceptual errors may alter inheritance relationships; and (3) restrictions on the format of the output model for ease of parsing. For example, as shown in Figure 2, the relationship `has` in the initial model is a misuse of inheritance as references. Hence, in the corrected model, MoRe changed it to a composition.

MoRe also directs the LLM to produce a reasoning analysis before outputting the revised model, ensuring this analysis is available for subsequent refinement. The entire

conversation history from the *name correction* phase remains visible during *concept correction*, enabling the LLM to preserve naming consistency and avoid logical contradictions.

3.3. Model Reduction

The third step of MoRe, called *model reduction*, aims to minimize redundant fragments—such as unnecessary classes, attributes, and references—in the domain model.

MoRe begins by extracting the domain model revised in the previous step from the LLM's latest response. It then instructs the LLM to review and simplify the model based on the domain requirements, guiding the reduction process step by step.

First, the LLM is asked to identify redundant elements. To support its judgment, MoRe provides explicit criteria:

- For a class, if it holds structural features or provides behavioral functionality to the system, it is not redundant.
- For an attribute, if it is mentioned in the requirements and is not derived, it is considered essential.
- For a reference, if it represents a structural relationship between objects that must be preserved (i.e., one that must be stored), rather than a transient runtime invocation.

Afterward, the LLM is requested to adjust and simplify the overly-designed fragments. For example, as shown in Figure 2, the attribute `address` is removed in the simplified model because it is not implied by the requirement.

Following the reduction step, MoRe queries the LLM again to detect and fix any grammatical errors that may remain in the domain model. This additional check is necessary because the reduction process can introduce syntax issues when the LLM adjusts the model structure. To ensure the output conforms to valid syntax, the prompt includes a carefully constructed example that illustrates the supported PlantUML grammar.

Finally, MoRe will invoke a customized parser to convert the domain model in PlantUML into an Ecore model because the last step of MoRe uses a rule-based checker that only works for Ecore models.

3.4. Rule-Based Fixing

While the previous two refinement steps are LLM-driven, the last step of MoRe is *rule-based fixing* with a model checker. The checker consists of a set of algorithmically implemented scanners, each dedicated to detecting a specific kind of modeling issue within an Ecore model. It should be noted that the issues checked and fixed in this step are concrete instances of the common issues listed in Table 1. If an issue identified by the checker is confirmed and resolved using a predefined strategy, MoRe directly fixes it by modifying the Ecore model. Otherwise, MoRe forwards an issue description—along with potential resolution strategies—to the LLM, requesting it to confirm the issue and select an appropriate repair strategy. MoRe will fix (or ignore) the issue according to the feedback of the LLM.

We now detail the issues supported by our rule-based checker and their corresponding fixing strategies, in the order in which they are processed.

1 The issue "id attribute" arises when a class contains both an attribute such as `xyz_id` and a reference to another class named `xyz`. In such cases, the attribute is considered an *unnecessary structural feature* and should be removed. To identify these issues, we defined a regular expression that matches potential *id attributes* in each class. For each candidate, we further verify whether a corresponding reference exists that can be paired with the attribute. Once an *id attribute* is confirmed, it is simply removed.

2 The issue "is-a association" occurs when an association is given a name that implies an inheritance relationship. This step serves as a double-check for the *misuse of association*

as inheritance. We employ a regular expression to detect any association named with terms such as `is`, `is-a`, `inherits`, `extends`, or similar variants. All *is-a associations* will be replaced with inheritance relationships.

3 The issue "empty child classes" arises when all child classes of a parent class are empty, i.e., they contain no attributes, participate in no references, and have no subclasses. This issue is a special case of *unnecessary classes*. To fix *empty child classes*, we introduce an enumeration to capture the subtype distinctions, add a type attribute to the parent class, and finally remove the now-redundant child classes.

4 The issue "unused enumeration" occurs when the model contains an enumeration that is never referenced. It is also a special case of an *unnecessary class*. For every enumeration, we traverse the model to check whether it is used as an attribute type. If not, we remove the enumeration from the model.

5 The issue "features to be pulled up" is detected when identical attributes appear across sibling classes, indicating they should be moved to the common superclass. *Features to be pulled up* are a special case of *unnecessary structural features*. The check involves traversing each parent's inheritance tree, collecting child class features, and flagging any feature that occurs in multiple siblings based on name and type. The fix removes these features from all sibling subclasses and adds them once to the parent class.

6 The issue "empty class" arises when a class is empty (see the issue *empty child classes*). While this is a special instance of *unnecessary classes*, we cannot automatically confirm that an empty class is truly redundant, because it may be the result of parsing errors during the PlantUML-to-Ecore conversion, which could have caused its original definitions to be lost. We must consult an LLM for confirmation. If the LLM determines that the class is redundant, MoRe will remove it.

7 The issue "derived reference" arises when a reference can be derived from two other references. It is a special case of *unnecessary structural features*. For example, if a `Professor` teaches a `Course` and the `Course` is enrolled by a `Student`, the direct relationship `Professor-->Student` may be derivable from the chain `Professor-->Course` and `Course-->Student` and could therefore be redundant. To check for this issue, we traverse all reference triples in the model and verify whether they form a transitive triangle. For each candidate triple, the LLM is asked to determine whether one of the references is semantically derived. If the LLM confirms that a reference is redundant, it is removed.

8 The issue "enumeration mirroring class hierarchy" arises when an enumeration is defined such that each of its literals semantically corresponds to a distinct subclass of a parent class. Detecting this issue cannot rely on simple string matching between enumeration literals and subclass names. Therefore, we employ LLM-assisted verification: for every parent class that contains an enumeration attribute, we present both the enumeration and the corresponding inheritance hierarchy to the LLM and request a judgment on whether they encode the same classification scheme. If so, we remove the enumeration and the attribute.

9 The issue "bidirectional reference pair" occurs when two classes are connected by two opposite references that represent the same logical relationship, resulting in *unnecessary features*. For example, in Figure 2, `departments` and `affiliatedTo` between `Company` and `Department` can be viewed as a bidirectional reference pair. Because we cannot determine whether two opposite references denote the same relationship based solely on structure, we send all candidate pairs of opposite references to the LLM and ask it to assess whether they represent the same semantic association and, if so, indicate which one can be safely removed. Finally, MoRe fixes this issue according to

the LLM's feedback (Since Ecore supports bidirectional references, MoRe turns a confirmed bidirectional reference pair into a properly configured bidirectional reference.).

10 The issue "duplicate reference pair" occurs when two classes are connected by two references that share the same direction and essentially represent the same semantic relationship, introducing unnecessary structural features. To check for this issue, we identify all reference pairs between the same two classes that have the same direction (i.e., share the same source and target), and then use the LLM to determine whether they are semantically equivalent and, if so, to indicate which reference should be removed.

4. Semantic Model Matcher

How LLM-generated models should be evaluated is an open issue. Existing research typically uses manual inspection, automatic Ecore model matchers, or LLM-based judgment. To minimize human bias and LLM instability, this paper proposes a semantic model matcher, namely, $SemMatch(\cdot)$, that realizes a deterministic matching algorithm.

First, we formally define a domain model m as a tuple $(CLS, ATT, REF, INH, ENM, LIT)$ of six model components, where

- CLS is the set of classes; each class has a class name and a set of structural features (i.e., attributes and references);
- ATT is the set of attributes; each attribute has a name and a (primitive) type; we assume that an attribute is always defined within an owner class;
- REF is the set of references (i.e., associations and compositions); each reference is defined by a name, a source class, and a target class; we assume that a reference is owned by its source class;
- INH is the set of inheritance relationships; each inheritance relationship is defined by a child class and a parent class; assume that an inheritance relationship is denoted as $c_1 \prec c_2$, where c_1 is a child class and c_2 is a parent class;
- ENM is the set of enumerations; each enumeration is defined by a name and some enumeration literals;
- LIT is the set of enumeration literals; each literal is defined by a name; we assume that every literal belongs to an enumeration.

The following functions are also provided to traverse the model.

- $features(c)$ returns the structural features defined in the class c.
- $literals(e)$ returns the literals defined in the enumeration e.
- $owner(f)$ returns the owner class/enumeration of the structural feature/literal f. If f is a structural feature, then $f \in features(owner(f))$; if f is an enumeration literal, then $f \in literals(owner(f))$.
- $src(f)$ and $tar(f)$ return the source and the target class of a reference f. Note that $src(f) = owner(f)$.

4.1. Matching Algorithm

$SemMatch(\cdot)$ takes two domain models m_0 and m as input and produces a match μ consisting of mappings from m_0 and m. Conceptually, μ can be regarded as a partial injective function: for each element x in m_0—which may be a class, attribute, reference, inheritance relationship, enumeration, or literal—the function yields either the matched element $\mu(x)$ in m, or the symbol $\perp$ if no corresponding element exists in m. We denote by $\mu^{-1}(y)$ the element in m_0 that matches a given element y in m (if such a match exists).

The overall framework of $SemMatch(\cdot)$ is heavily inspired by the state-of-the-art open-source model matcher EMF Compare https://eclipse.dev/emfcompare/ (accessed on 12 February 2026). However, EMF Compare generally relies on unique id values or edit

distances to determine whether two elements are matched. Since an LLM may generate synonyms that are literally different from the expected name but semantically equivalent, EMF Compare cannot be applied straightforwardly.

Instead, $SemMatch(\cdot)$ matches two models based on their semantic embedding vectors. The main matching algorithm is presented in Algorithm 1. Assume that S is a meta-variable ranging over {CLS, ATT, REF, INH, ENM, LIT}. Given $x \in S_{m_0}$, the key idea of this algorithm is to use a similarity function sim to find the **closest match** y within S_m—x and y must be matchable and $sim(x, y)$ must achieve the highest similarity.

Algorithm 1 Algorithm of semantic model matcher $SemMatch(m_0, m)$

$m_0 \equiv (CLS_{m_0}, ATT_{m_0}, REF_{m_0}, INH_{m_0}, ENM_{m_0}, LIT_{m_0})$
$m \equiv (CLS_m, ATT_m, REF_m, INH_m, ENM_m, LIT_m)$
$\mu \leftarrow \emptyset$
for S in {CLS, ATT, REF, INH, ENM, LIT} **do**
    for x in S_{m_0} **do**
        $y \leftarrow findClosest(x, m_0, m, S, \mu,$ **true**$)$
        if $y \neq \perp$ **then**
            $\mu \leftarrow \mu \cup \{(x, y)\}$
        end if
    end for
end for
return μ

$findClosest(\cdot)$, as defined in Algorithm 2, is responsible for matching x. First, it constructs a candidate set $candY$ from every unmatched y in S_m that is matchable to x and has a similarity to x exceeding τ_S. Second, it sorts $candY$ in descending order of similarity, producing the list $sortedY$. It then examines the first element y in $sortedY$ and verifies whether x is also the closest unmatched element for y. If this condition holds, y is returned as the closest match for x. Otherwise, the algorithm proceeds to check the next y in $sortedY$.

Algorithm 2 Algorithm of $findClosest(x, m_0, m, S, \mu, dbchk)$

$candY \leftarrow \{y \mid y \in S_m \wedge \mu^{-1}(y) = \perp \wedge canMatch(x, y, \mu) = \texttt{true} \wedge sim(x, y) > \tau\}$
$sortedY \leftarrow$ sort $candY$ in descending order by $sim(x, y)$
for $y \in sortedY$ **do**
    if dbchk **then**
        $yBest \leftarrow findClosest(y, m, m_0, S, \mu^{-1},$ **false**$)$
        if $x = yBest$ **then**
            return y
        end if
    else
        return y
    end if
end for
return $\perp$

The function $canMatch(x, y, \mu)$ checks whether x and y can be matched, as shown in Algorithm 3. For classes and enumerations, the function always returns true. For attributes and literals, the function returns true only when $\mu(owner(x)) = owner(y)$ holds, that is to say, two attributes/literals can be matched only when their owner classes/enumerations are already matched. For two references, the function returns true only when the source and target classes of x and y are already matched, i.e., $\mu(src(x)) = src(y) \wedge \mu(tar(x)) = tar(y)$ or $\mu(src(x)) = tar(y) \wedge \mu(tar(x)) = src(y)$. For two inheritance relationships, say $x = c_1 \prec c_2$ and $y = c_3 \prec c_4$, the function returns true if and only if $\mu(c_1) = c_3 \wedge \mu(c_2) = c_4$ holds, that

is to say, two inheritance relationships can be matched only when their child and parent classes are already matched.

Algorithm 3 Algorithm of $canMatch(x, y, \mu)$

if x, y are classes or enumerations **then**
  return true
else if x, y are attributes or literals **then**
  return $owner(x) = owner(y)$
else if x, y are references **then**
  if $\mu(src(x)) = src(y) \wedge \mu(tar(x)) = tar(y))$ **then**
    return true
  else if $\mu(src(x)) = tar(y) \wedge \mu(tar(x)) = src(y)$ **then**
    return true
  else
    return false
  end if
else if x, y are inheritance relationships **then**
  $x \equiv c_1 \prec c_2$
  $y \equiv c_3 \prec c_4$
  return $\mu(c_1) = c_3 \wedge \mu(c_2) = c_4$
else
  return false
end if

The function $sim(x, y)$ serves as the core of the matching process. Its algorithm is presented in Algorithm 4. First, it generates two strings from x and y using $toStr(\cdot)$. Second, it computes the embedding vectors from the strings using $embedding(\cdot)$. In implementation, we used a sentence transformer called MiniLM-L6-V2 [36] to compute the embedding vectors. Finally, the function returns the cosine similarity between the two vectors.

Algorithm 4 Similarity function $sim(\cdot)$

$\text{str}_x \leftarrow \text{toStr}(x)$
$\text{str}_y \leftarrow \text{toStr}(y)$
$\vec{v}_x \leftarrow \text{embedding}(\text{str}_x)$
$\vec{v}_y \leftarrow \text{embedding}(\text{str}_y)$
return $\cos(\vec{v}_x, \vec{v}_y)$

The behavior of the function $toStr(x)$ varies according to the type of the input value x.

- If x is a class, then $toStr(x)$ returns a string concatenation of the constant string `"class"`, the name of x, and $toStr(f)$ for every feature f in $features(x)$.
- If x is an attribute, then $toStr(x)$ returns a string concatenation of the attribute type's name and the attribute's name.
- If x is a reference, then $toStr(x)$ returns a string concatenation of the source class's name, the reference's name , and the target class's name.
- If x is an enumeration, then $toStr(x)$ returns a string concatenation of the constant string `"enum"`, the name of x, and $toStr(l)$ for every literal l in $literals(x)$.
- If x is an enumeration literal, then $toStr(x)$ simply returns the literal name.
- If x is an inheritance relationship, then $toStr(x)$ returns a constant string `"inheritance"` because inheritance relationships are matched solely based on their child and parent classes, and their embedding similarity does not affect the matching process.

4.2. Selection of Hyper-Parameters

$SemMatch(\cdot)$ uses six hyper-parameters τ_{CLS}, τ_{ATT}, τ_{REF}, τ_{INH}, τ_{ENM}, and τ_{LIT} as the lower bounds of similarity for matching classes, attributes, references, inheritance

relationships, enumerations, and literals, respectively. To select appropriate values for these hyper-parameters, we conducted a pilot experiment with the help of the reference models in our problem set. Note that we fix $\tau_{\text{INH}} = 1.0$, as inheritance matching depends only on class matching, not on embedding similarity.

We illustrate the threshold determination process using τ_{CLS} as an example. For each class c in reference models, we first prompt an LLM to generate 20 semantically equivalent mutants of c by replacing the class name with synonyms and altering its structural features, then compute the similarity $sim(c, c')$ for each mutant c'. Concurrently, we compute the similarity between c and mutants derived from other classes in the same model. The threshold τ_{CLS} is then chosen to optimally separate the higher similarity scores (from equivalent mutants) from the lower ones (from non-equivalent mutants). By repeating this process for attributes, references, enumerations, and literals, we can determine their hyper-parameters.

We performed the selection procedure on DeepSeek V3.2 and GLM-5. Figure 3 depicts how F1 scores vary across different threshold values. Based on Figure 3, we observe two main findings: (1) The optimal threshold varies across tasks. For instance, the best thresholds for classes and enumerations lie between 0.6 and 0.7, whereas for attributes, the optimal threshold falls between 0.4 and 0.5. (2) For a given task, different LLMs exhibit similar performance trends, though their absolute results differ. For example, DeepSeek achieved the highest F1 score for classes when τ_{CLS} is set to 0.6, while for GLM-5, the optimal τ_{CLS} should be around 0.65.

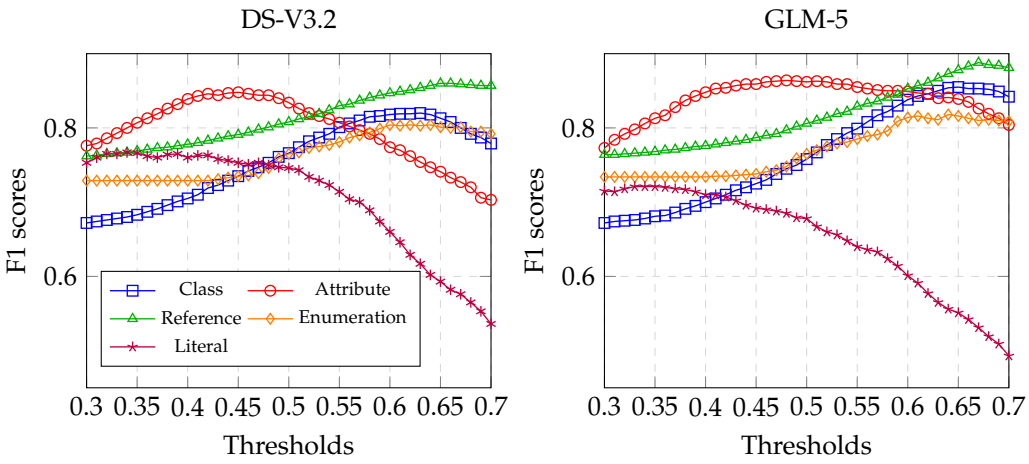


Figure 3. Tests of hyper-parameters based on DeepSeek V3.2 and GLM-5.

To select the optimal threshold, we first analyzed the results from DeepSeek and GLM-5 shown in Figure 3. For a given task, we took the x-value at which the corresponding curve reached its highest point as the optimal threshold for that LLM. Finally, the smaller of the two optimal thresholds from the two LLMs was chosen as the value for the respective hyper-parameter. Since $SemMatch(\cdot)$ uses these thresholds to filter implausible matches, a lower value risks pairing semantically dissimilar elements. Generally, poorer-performing approaches benefit from lower thresholds.

Finally, we chose the following thresholds: $\tau_{CLS} = 0.6$, $\tau_{ATT} = 0.45$, $\tau_{REF} = 0.65$, $\tau_{ENM} = 0.6$, and $\tau_{LIT} = 0.34$.

5. Experimental Design

5.1. Goal and Research Questions

The goal of this section is to evaluate MoRe with respect to its effectiveness in generating domain models from the perspective of academic research in the context of LLM-driven domain modeling.

Based on the research goal, we set up three research questions:

RQ1 Does MoRe effectively improve the quality of generated domain models?

Rationale. **RQ1** examines whether MoRe achieves its core objective: improving the quality of LLM-generated domain models. It verifies if the framework produces measurable gains—a key requirement for academic relevance and practical adoption.

RQ2 To what extent does the performance of MoRe vary across different LLMs?

Rationale. Because MoRe relies on underlying LLMs, its performance may vary across different models. **RQ2** assesses this sensitivity to determine whether MoRe is robust and generalizable, or if its effectiveness depends heavily on the choice of LLM.

RQ3 How does each component of MoRe contribute to its overall effectiveness?

Rationale. MoRe integrates several components into a multi-step refinement pipeline. **RQ3** conducts an ablation study to quantify the contribution of each component, revealing which stages are most influential and how they interact.

5.2. Problem Set

To evaluate an LLM-based domain model generation approach, we required a set of domain modeling problems, each consisting of a textual domain requirement and a reference domain model.

We constructed a problem set of 30 domain modeling problems by collecting textual requirements from existing literature [25,31,37]. These requirements cover a wide range of domains, such as business, transportation, entertainment, education, technology, construction, finance, and others. For each requirement, we carefully reviewed the text and corrected any grammatical errors to ensure clarity and consistency. We also adjusted the requirements to eliminate any ambiguous or irrelevant information that could potentially mislead the model generation process. Since none of these problems came with a formally defined reference model in Ecore format, we manually created reference models for all the requirements based on the requirements and following the best practices of object-oriented modeling. To ensure the quality of the reference models, two authors independently reviewed each model (with an inter-rater agreement of 86.7%) and resolved any discrepancies through discussion until a consensus was reached.

The general statistics of the problem set are summarized in Table 2.

Table 2. General statistics of the problem set.

Domain	#Probs.	#Sent.	#Wrd.	#Cls.	#Attr.	#Ref.	#Inh.	#Enum.	#Lit.
Business	6	8.2	99.7	6.0	10.2	4.7	2.0	1.2	3.0
Transportation	6	12.2	167.2	6.7	11.2	5.7	1.7	0.9	2.2
Entertainment	5	15.2	228.6	5.0	10	6.6	0.0	1.0	3.2
Education	4	10.3	118.0	5.3	7.8	3.3	1.8	2.0	4.3
Technology	3	11.7	165.0	8.0	5.0	5.0	5.0	0.3	0.7
Construction	2	8.0	110.0	12.0	6.0	11.5	4.5	0.0	0.0
Finance	2	8.5	145.5	5.0	10.5	3.5	1.0	0.5	1.5
Others	2	11.0	137.0	7.0	6.0	4.0	3.0	2.0	5.0

Probs. = Problems; Sent. = Sentences; Wrd. = Words; Cls. = Classes; Attr. = Attributes; Ref. = References; Inh. = Inheritance relationships; Enum. = Enumerations; Lit. = Enumeration literals. All values (except for #Probs.) are averages across all problems in a domain.

5.3. Metrics

To measure the quality of a generated domain model, we propose adopting the metric F1 score, which is defined by Equation (1). We compute one F1 score for each component, i.e., classes, attributes, references, inheritance relationships, enumerations, and literals.

$$F1 = \frac{2 \times precision \times recall}{precision + recall} \tag{1}$$

The sub-metric *precision* measures the consistency of a generated model with respect to the reference model, while *recall* measures how comprehensively the reference model is captured by the generated model. Given a generated model m and a reference model m_0, our core idea is to establish a matching μ from m_0 to m.

Suppose that m_0 is defined as $(CLS_{m_0}, ATT_{m_0}, REF_{m_0}, INH_{m_0}, ENM_{m_0}, LIT_{m_0})$ and m is defined as $(CLS_m, ATT_m, REF_m, INH_m, ENM_m, LIT_m)$. Let S range over {CLS, ATT, REF, INH, ENM, LIT}. Precision and recall are defined as Equations (2) and (3).

$$precision_S = \frac{|\{y | y \in S_m \wedge \mu^{-1}(y) \neq \perp\}|}{|S_m|} \tag{2}$$

$$recall_S = \frac{|\{x | x \in S_{m_0} \wedge \mu(x) \neq \perp\}|}{|S_{m_0}|} \tag{3}$$

The two sub-metrics can be generalized to a reference model set M_0 and a generated model set M as follows.

$$precision_S = \frac{\sum_{m \in M} |\{y | y \in S_m \wedge \mu^{-1}(y) \neq \perp\}|}{\sum_{m \in M} |S_m|} \tag{4}$$

$$recall_S = \frac{\sum_{m_0 \in M_0} |\{x | x \in S_{m_0} \wedge \mu(x) \neq \perp\}|}{\sum_{m_0 \in M_0} |S_{m_0}|} \tag{5}$$

The F1 score can also be naturally generalized. In our evaluation, we calculated F1 scores for model sets.

5.4. Selection of Baselines and LLMs

To evaluate the effectiveness of MoRe, we selected the following baselines for comparison:

- Simple: This baseline directly prompts an LLM to generate a domain model from the requirement without any refinement using the same prompt as used for initial generation. It serves as a lower bound for evaluating the improvement brought by MoRe.
- Iterative: This baseline is proposed by Yang et al. [32] and represents a state-of-the-art approach for LLM-driven domain modeling.

To investigate the contribution of each refinement step, we further constructed three variants of MoRe by semantically disabling each refinement step:

- MoRe-MC is a variant of MoRe that disables the model correction step, i.e., it only performs model reduction and rule-based fixing.
- MoRe-MR is a variant of MoRe that disables the model reduction step, i.e., it only performs model correction and rule-based fixing.
- MoRe-RF is a variant of MoRe that disables the rule-based fixing step, i.e., it only performs model correction and model reduction.

We selected four open-source LLMs from two representative families: DeepSeek and Qwen3 Coder. Table 3 summarizes their specifications.

Table 3. Selected Large Language Models.

ID	Model	# Parameters	Release Date
DS-V3.2	DeepSeek V3.2	671B	December 2025
DS-R1-8B	DeepSeek-R1-Distill-Llama-8B	8B	February 2025
Q3C-480B	Qwen3-Coder-480B-A35B-Instruct	480B	July 2025
Q3C-30B	Qwen3-Coder-30B-A3B-Instruct	30B	August 2025

6. Results

We applied six approaches—MoRe, Simple, Iterative, MoRe-MC, MoRe-MR, and MoRe-RF—to generate domain models from the 30 domain requirements in our problem set. MoRe, Simple, and Iterative were tested on the four selected LLMs, while MoRe-MC, MoRe-MR, and MoRe-RF were tested on DeepSeek V3.2. For each generated model and its corresponding reference model, we computed a match μ using $SemMatcher(\cdot)$. Based on these matches, we derived an F1 score for each set of generated models. To ensure statistical reliability, the entire process was repeated five times, and the average results were calculated. Appendix B presents two concrete examples of the model generation.

Table 4 presents the F1 scores for three model generation approaches to be compared—Simple, Iterative, and MoRe—across six model components (CLS, ATT, REF, INH, ENM, LIT) using four different LLMs. The first column indicates the approach, while the second column specifies the LLM used for generation. The subsequent columns report the F1 scores for each model component, along with their standard deviations across 5 generations.

Table 4. Comparison over different domain model generation approaches.

Approach	LLM	F1 Score					
		CLS	ATT	REF	INH	ENM	LIT
Simple	DS-V3.2	$0.774_{\pm 0.01}$	$0.662_{\pm 0.004}$	$0.51_{\pm 0.006}$	$0.59_{\pm 0.023}$	$0.726_{\pm 0.027}$	$0.718_{\pm 0.028}$
	DS-R1-8B	$0.728_{\pm 0.007}$	$0.654_{\pm 0.029}$	$0.312_{\pm 0.032}$	$0.396_{\pm 0.134}$	$0.49_{\pm 0.083}$	$0.43_{\pm 0.082}$
	Q3C-480B	$0.756_{\pm 0.019}$	$0.728_{\pm 0.019}$	$0.492_{\pm 0.079}$	$0.552_{\pm 0.016}$	$0.65_{\pm 0.111}$	$0.612_{\pm 0.09}$
	Q3C-30B	$0.77_{\pm 0.021}$	$0.616_{\pm 0.008}$	$0.44_{\pm 0.018}$	$0.576_{\pm 0.034}$	$0.61_{\pm 0.069}$	$0.502_{\pm 0.044}$
Iterative	DS-V3.2	$0.72_{\pm 0.015}$	$0.602_{\pm 0.015}$	$0.36_{\pm 0.023}$	$0.554_{\pm 0.045}$	$0.624_{\pm 0.081}$	$0.582_{\pm 0.079}$
	DS-R1-8B	$0.608_{\pm 0.016}$	$0.432_{\pm 0.018}$	$0.226_{\pm 0.016}$	$0.164_{\pm 0.046}$	$0.356_{\pm 0.096}$	$0.28_{\pm 0.05}$
	Q3C-480B	$0.676_{\pm 0.027}$	$0.548_{\pm 0.023}$	$0.354_{\pm 0.019}$	$0.474_{\pm 0.052}$	$0.472_{\pm 0.126}$	$0.456_{\pm 0.116}$
	Q3C-30B	$0.358_{\pm 0.09}$	$0.422_{\pm 0.041}$	$0.222_{\pm 0.056}$	$0.346_{\pm 0.093}$	$0.198_{\pm 0.038}$	$0.196_{\pm 0.05}$
MoRe	DS-V3.2	$0.828_{\pm 0.012}$	$0.696_{\pm 0.01}$	$0.544_{\pm 0.023}$	$0.708_{\pm 0.047}$	$0.768_{\pm 0.029}$	$0.756_{\pm 0.037}$
	DS-R1-8B	$0.716_{\pm 0.024}$	$0.624_{\pm 0.043}$	$0.324_{\pm 0.024}$	$0.194_{\pm 0.128}$	$0.3_{\pm 0.126}$	$0.254_{\pm 0.115}$
	Q3C-480B	$0.822_{\pm 0.026}$	$0.75_{\pm 0.015}$	$0.564_{\pm 0.028}$	$0.618_{\pm 0.049}$	$0.76_{\pm 0.021}$	$0.726_{\pm 0.041}$
	Q3C-30B	$0.832_{\pm 0.007}$	$0.748_{\pm 0.019}$	$0.574_{\pm 0.022}$	$0.646_{\pm 0.048}$	$0.77_{\pm 0.021}$	$0.75_{\pm 0.029}$

F1 scores across metrics range as follows: CLS 0.358–0.832; ATT 0.422–0.750; REF 0.222–0.574; INH 0.164–0.708; ENM 0.198–0.770; LIT 0.196–0.756. The highest scores for each metric are achieved by MoRe with specific LLMs: CLS by DS-V3.2 (0.828) and Q3C-30B (0.832); ATT by Q3C-480B (0.750) and Q3C-30B (0.748); REF by Q3C-30B (0.574); INH by DS-V3.2 (0.708); ENM by DS-V3.2 (0.768) and Q3C-30B (0.770); LIT by DS-V3.2 (0.756) and Q3C-30B (0.750). Standard deviations range from ± 0.004 (ATT under Simple) to ± 0.134 (INH under Simple for DS-R1-8B), with most within ± 0.01–± 0.05 and broader intervals primarily for INH and ENM.

Table 5 reports the two-tailed p-values obtained from two different significance testing methods (the Mann-Whitney U test and the permutation test), comparing MoRe with baseline approaches across different evaluation metrics. The left panel shows results from the Mann-Whitney U test, while the right panel presents results from the permutation test (with 1,000,000 permutations), which provides a more robust alternative by empirically estimating the null distribution through random reassignment of group labels.

Table 5. Significance test results (p-values) of baselines and MoRe.

Baseline	LLM	Mann-Whitney U						Permutation Test (with 1,000,000 Permutations)					
		CLS	ATT	REF	INH	ENM	LIT	CLS	ATT	REF	INH	ENM	LIT
Simple	DS-V3.2	0.012	0.010	0.018	0.021	0.107	0.243	0.008	0.004	0.024	0.015	0.143	0.169
	DS-R1-8B	0.584	0.287	1.000	0.074	0.059	0.056	0.618	0.313	0.640	0.087	0.034	0.035
	Q3C-480B	0.019	0.087	0.105	0.026	0.045	0.036	0.009	0.151	0.121	0.017	0.032	0.019
	Q3C-30B	0.011	0.010	0.011	0.115	0.012	0.012	0.006	0.007	0.008	0.074	0.007	0.008
Iterative	DS-V3.2	0.011	0.011	0.012	0.008	0.020	0.021	0.008	0.001	0.008	0.008	0.016	0.016
	DS-R1-8B	0.012	0.012	0.012	0.746	0.462	0.834	0.008	0.008	0.007	0.836	0.508	0.695
	Q3C-480B	0.011	0.011	0.012	0.012	0.012	0.012	0.003	0.003	0.007	0.006	0.001	0.006
	Q3C-30B	0.012	0.011	0.012	0.008	0.012	0.012	0.006	0.006	0.008	0.007	0.008	0.008

The p-values from comparisons between MoRe and the Iterative baseline are low in most cases, indicating statistical significance. Mann-Whitney U tests show $p < 0.05$ for all LLMs and metrics except for DS-R1-8B on INH ($p = 0.746$), ENM ($p = 0.462$), and LIT ($p = 0.834$). Permutation tests yield even smaller p-values elsewhere and identical non-significant results for that model (INH $p = 0.836$, ENM $p = 0.508$, LIT $p = 0.695$).

Against the Simple baseline, most metrics also show $p < 0.05$, with the Qwen3 series yielding $p < 0.05$ in both tests across nearly all metrics. Non-significant results again appear for DS-R1-8B, with $p > 0.05$ for CLS, ATT, REF and near-threshold values for ENM (0.059/0.034) and LIT (0.056/0.035), with the permutation test generally producing slightly smaller p-values.

Table 6 presents the results of an ablation study conducted using DS-V3.2 and Q3C-480B. It is organized with "LLM" and "Approach" as the row variables and six evaluation metrics (CLS, ATT, REF, INH, ENM, LIT) as column variables. Four distinct approaches are compared: MoRe, and three ablated variants denoted as MoRe-MC, MoRe-MR, and MoRe-RF.

Table 6. Results of ablation study based on DeepSeek V3.2 and Qwen3-Coder-480B-A35B-Instruct.

LLM	Approach	F1 Score					
		CLS	ATT	REF	INH	ENM	LIT
DS-V3.2	MoRe	$0.828_{\pm 0.012}$	$0.696_{\pm 0.01}$	$0.544_{\pm 0.023}$	$0.708_{\pm 0.047}$	$0.768_{\pm 0.029}$	$0.756_{\pm 0.037}$
	MoRe-MC	$0.822_{\pm 0.013}$	$0.702_{\pm 0.012}$	$0.546_{\pm 0.022}$	$0.638_{\pm 0.043}$	$0.756_{\pm 0.057}$	$0.746_{\pm 0.050}$
	MoRe-MR	$0.78_{\pm 0.023}$	$0.592_{\pm 0.018}$	$0.476_{\pm 0.022}$	$0.628_{\pm 0.082}$	$0.672_{\pm 0.083}$	$0.622_{\pm 0.073}$
	MoRe-RF	$0.81_{\pm 0.018}$	$0.666_{\pm 0.017}$	$0.554_{\pm 0.027}$	$0.674_{\pm 0.037}$	$0.716_{\pm 0.041}$	$0.686_{\pm 0.045}$
Q3C-480B	MoRe	$0.822_{\pm 0.026}$	$0.75_{\pm 0.015}$	$0.564_{\pm 0.028}$	$0.618_{\pm 0.049}$	$0.76_{\pm 0.021}$	$0.726_{\pm 0.041}$
	MoRe-MC	$0.814_{\pm 0.016}$	$0.734_{\pm 0.014}$	$0.55_{\pm 0.03}$	$0.664_{\pm 0.039}$	$0.768_{\pm 0.05}$	$0.746_{\pm 0.038}$
	MoRe-MR	$0.802_{\pm 0.021}$	$0.606_{\pm 0.033}$	$0.54_{\pm 0.018}$	$0.71_{\pm 0.017}$	$0.748_{\pm 0.03}$	$0.69_{\pm 0.033}$
	MoRe-RF	$0.808_{\pm 0.021}$	$0.694_{\pm 0.017}$	$0.534_{\pm 0.014}$	$0.622_{\pm 0.061}$	$0.742_{\pm 0.029}$	$0.688_{\pm 0.015}$

For DS-V3.2, MoRe leads on CLS (0.828), INH (0.708), ENM (0.768), and LIT (0.756). The MC variant tops ATT (0.702). The RF variant tops REF (0.554). The MR variant is the lowest, for example scoring 0.78 on CLS. For Q3C-480B, MoRe leads on CLS (0.822), ATT (0.75), and REF (0.564). The MC variant tops ENM (0.768) and LIT (0.746). The MR variant tops INH (0.71). This model shows higher ATT scores overall. Across LLMs, MoRe delivers top or near-top scores on most metrics.

We performed a significance test to compare MoRe with its ablation variants, and the results show that, in most cases, the differences between the variants and MoRe were not significant. Since the F1 score is the harmonic mean of precision and recall, this averaging process may have affected the significance level of the results. To analyze the impact of different variants in depth, we conducted separate significance tests for precision and recall; the specific results are shown in Table 7. The cell's value is the p-value from a Mann-Whitney U test, indicating the significance of the difference between a variant and MoRe for that category. We also fill all cells with $p < 0.1$ in different colors to indicate the

direction of the effect: red cells indicate a decrease (i.e., the variant's result is lower than MoRe's result), and blue cells indicate an increase.

Table 7. Significance test results (p-values) of ablation study.

LLM	Variant	p-Values for Precision						p-Values for Recall					
		CLS	ATT	REF	INH	ENM	LIT	CLS	ATT	REF	INH	ENM	LIT
DS-V3.2	MoRe-MC	0.737	0.519	0.671	0.012	0.665	1.000	0.828	0.589	0.833	0.398	0.745	0.917
	MoRe-MR	0.011	0.011	0.008	0.011	0.012	0.012	0.090	0.163	0.236	0.526	0.915	1.000
	MoRe-RF	0.026	0.090	0.168	0.035	0.092	0.074	0.753	1.000	0.093	0.138	0.175	0.246
Q3C-480B	MoRe-MC	0.023	0.087	0.112	0.008	0.173	0.674	0.011	0.651	0.036	0.203	0.828	0.833
	MoRe-MR	0.138	0.012	0.071	0.027	0.012	0.032	0.915	0.056	0.343	0.832	0.072	0.036
	MoRe-RF	0.916	0.012	0.138	0.841	0.753	0.116	0.014	0.056	0.195	1.000	0.589	0.753

p means a significant decrease ($p < 0.05$); p means a borderline significant decrease ($0.05 \leq p < 0.1$); p means a significant increase ($p < 0.05$); p means a borderline significant increase ($0.05 \leq p < 0.1$).

For DS-V3.2, MoRe-MR and MoRe-RF significantly decrease precision across most categories: MoRe-MR shows $p < 0.05$ in 5/6 metrics, MoRe-RF in 3/6 with borderline decreases ($0.05 \leq p < 0.1$) in the remaining three. MoRe-MC shows only INH precision decreasing significantly ($p = 0.012$). Recall is largely unaffected, with only MoRe-MR showing a borderline CLS decrease ($p = 0.090$).

For Q3C-480B, MoRe-MC yields significant precision increases for CLS ($p = 0.023$) and INH ($p = 0.008$), but significant recall decreases for CLS ($p = 0.011$) and REF ($p = 0.036$). MoRe-MR shows significant precision decreases for ATT, ENM, LIT ($p < 0.05$) and borderline REF decrease ($p = 0.071$), with significant LIT recall increase ($p = 0.036$). MoRe-RF causes significant precision decrease only for ATT ($p = 0.012$), but significant recall decreases for CLS ($p = 0.014$) and borderline ATT decrease ($p = 0.056$).

7. Answers to Research Questions

Answer to RQ1

According to Table 4, MoRe consistently achieves the highest or near-highest F1 scores across most categories (CLS, ATT, REF, INH, ENM, LIT) for the three most capable LLMs (DS-V3.2, Q3C-480B, and Q3C-30B). The significance test results in Table 5 also confirm that MoRe substantially improves the quality of the generated models, particularly improving the generation of classes, attributes, and associations/compositions, compared to the baseline methods for most of the LLMs we tested.

MoRe outperforms Simple approach by introducing structured refinement steps. While Simple approach generates a model in one shot, it is prone to omissions, inconsistent relationships, and superficial understanding. MoRe's multi-stage process—generate, critique, and refine—allows it to identify and correct these specific errors post-hoc.

The stark superiority of MoRe over Iterative baseline empirically demonstrates the vulnerability of an unstructured, naive multi-step generation process. Iterative approach, which constructs a domain model incrementally, suffers from conceptual drift, error propagation, and hallucination. In contrast, MoRe provides a controlled refinement loop with a specific, error-focused critique, preventing aimless drift and anchoring improvements to the initial valid structure.

Notably, Iterative suffers from catastrophic performance drops on DS-R1-8B and Q3C-30B. While MoRe also drops for DS-R1-8B, its performance with Q3C-30B remains comparable to larger models. This indicates that MoRe is more robust than Iterative when using mid-sized LLMs. One possible explanation is that MoRe uses clearer, shorter prompts than Iterative, making them easier for mid-sized LLMs to follow. The rule-based fixes in MoRe also reduce its reliance on the capabilities of LLMs.

While MoRe excels at fixing redundancy and inconsistencies, it may be less effective if the initial generation is fundamentally flawed or misunderstands the core domain, as refinement operates on an existing structure.

Answer to RQ2

Table 4 further highlights that MoRe's performance varies considerably across different LLMs. The most pronounced improvements over Simple are observed with Q3C-30B, which shows substantial gains across nearly all evaluation categories. In contrast, both DS-V3.2 and Q3C-480B exhibit more moderate, yet consistent, performance improvements.

A possible explanation for these results is that Q3C-30B occupies a unique performance sweet spot: it is sufficiently large to comprehend the abstractions and instructions required for domain modeling, yet small enough to benefit disproportionately from MoRe 's step-by-step scaffolding. While larger models (DS-V3.2 and Q3C-480B) also improve consistently under MoRe—underscoring the general utility of the self-refinement strategy—their gains are more modest. This implies that they have already internalized many reasoning patterns. Conversely, the smallest model appears overwhelmed by the decomposition, resulting in performance degradation.

The smaller DS-R1-8B model yields mixed outcomes with notable degradation on several structural components (INH, ENM, LIT). This suggests that MoRe, which involves complex reasoning and long, intricate prompts, requires a minimal level of LLM capability. Specifically, the LLM must be able to (1) effectively process and follow long, multi-constraint instructions, and (2) generate outputs with precise syntax adherence (e.g., valid PlantUML code). The execution logs reveal that DS-R1-8B frequently generated syntactically invalid PlantUML and failed to properly follow our instructions.

For AI-assisted domain modeling, selecting an LLM based solely on generic benchmark scores is inadequate. Practitioners should instead evaluate candidate LLMs in conjunction with the intended approaches (e.g., MoRe) to assess the task performance. A medium-sized LLM like Q3C-30B, when paired with MoRe, may surpass larger models used with simpler prompts, offering a favorable balance of cost and accuracy for many projects. Even when access to very large language models is limited—for instance, in environments with restricted external connectivity—high-quality domain modeling remains achievable by applying structured, decompositional prompting to capable mid-sized LLMs.

Answer to RQ3

Based on Table 6, removing any component generally degrades the overall performance of MoRe. Their integration enables MoRe to achieve stable, well-rounded performance across different backbone LLMs. MoRe consistently achieves top or near-top performance across most metrics for both models, with ablation variants only outperforming on individual metrics.

However, as shown in Table 7, each component affects performance in distinct ways and to varying degrees. Overall, the components in MoRe contribute to a notable improvement in precision, albeit with a slight trade-off in recall. This outcome is expected, as the refinements introduced in MoRe are primarily designed to eliminate errors, redundancies, and inconsistencies in modeling, thereby enhancing precision. Among all components, *model reduction* exerts the most substantial impact on overall performance—particularly on precision—followed by *rule-based fixing*. Moreover, the effects of individual components vary across different LLMs. For instance, with DS-V3.2, all components enhance precision; however, for Q3C-480B, *model correction* reduces precision in CLS and INH while improving recall in CLS and REF. These variations reflect differences in architecture and capability across LLMs and further underscore that it is the integration of all components that endows MoRe with robust performance.

These results confirm that the effectiveness of MoRe stems from the synergistic integration of complementary components. This aligns with the inherent nature of object-oriented modeling, where accurate domain modeling requires balancing consistency, conciseness, extensibility, and flexibility. Each component addresses a distinct dimension of this challenge: *model reduction* captures design conciseness, *model correction* enforces structural and hierarchical validity, and *rule-based fixing* refines local logical conformance. Their combination allows MoRe to manage these trade-offs dynamically instead of over-optimizing for one criterion.

The ablation patterns further reveal that MoRe supports task-aware adaptation based on modeling priorities. Variants without certain components can excel on specific metrics such as ATT, REF, ENM, LIT, or INH, depending on the backbone LLM. For instance, if a user works with DeepSeek V3.2 and wants to maximize the quality of reference generation, he/she may try MoRe-RF. This suggests that MoRe can be deployed in its full configuration for balanced, robust performance, or adjusted to emphasize particular modeling properties when domain requirements warrant.

8. Discussion and Limitations

8.1. Discussion

PlantUML Parser

MoRe employs a customized PlantUML parser to convert a PlantUML model into an Ecore model. The parser realizes a subset of PlantUML grammar rules for essential domain modeling concepts, including the rules for *classes*, *attributes*, *enumerations* with *literals*, *inheritance*, and *binary associations/compositions*. As explained in Section 2, we ignore other constructs, such as operations, n-ary associations, and stereotypes, to be consistent with existing research efforts [20–22]. The parser also supports various grammar variants for the same construct. For example, both `t a` and `a:t` are accepted as an attribute `a` with type `t` by the parser.

The official PlantUML parser is unsuitable for our purposes, as it halts entirely on any grammar error, producing only an empty result. Our parser, by contrast, is fault-tolerant: it skips erroneous or unsupported constructs and continues processing. Admittedly, this robustness comes at the cost of potential information loss, as both valid out-of-scope constructs and syntactically malformed ones are discarded. Furthermore, our rule-based fixing introduces similar trade-offs. Specifically, there is a risk that LLMs might misjudge bidirectional reference pairs (targeted by scanner #9), which could negatively impact REF scores. We acknowledge this limitation, and quantifying the precise impact of scanner #9's false positives, particularly in the context of rule-based fixing, is part of our future work. However, we contend that generating a syntactically valid model is a non-negotiable prerequisite for any practical domain model generation approach, making this design choice both necessary and reasonable.

Advantages and Disadvantages of MoRe

MoRe focuses on object-oriented domain modeling. Its refinement steps minimize errors, redundancies, and inconsistencies in domain models. Tables 4 and 5 demonstrate that MoRe can improve the quality of generated models.

As Table 7 shows, MoRe generally improves precision but has little impact on recall. Thus, if the initial generation omits many elements (i.e., a lower recall), our approach is less effective. However, we believe MoRe's refinement steps can be integrated into other approaches. For instance, combining them with an approach that improves recall could maximize overall performance.

A further recognized limitation is that MoRe is currently confined to domain modeling and does not yet facilitate architectural and behavioral design, which necessitates the use of multiple system views [23]. Extending the approach to encompass full class diagram generation and additional diagram types will be part of our future work.

Completeness and Representativeness of Anti-Patterns

MoRe relies on a set of anti-patterns derived from the common issues listed in Table 1. We do not claim that these anti-patterns form a complete set.

To validate the representativeness of our anti-patterns, we compared them against common errors documented in existing literature [38–40] that are summarized from hand-crafted models. We found a significant overlap. For example, "Presence of derived or redundant attribute(s)" and "Inappropriate naming of classes and associations" proposed in [38] correspond to "Unnecessary structural features", "Using generic name", and "Empty name" in Table 1; "relations' type errors" proposed in [40] correspond to "Misuse of association as inheritance" and "Misuse of inheritance as reference". This alignment indicates that our identified anti-patterns are not biased. Instead, they highlight a natural consequence of LLM training: these LLMs, having learned from human-crafted examples, inevitably internalize and reproduce similar error patterns.

8.2. Threats to Validity

This section systematically identifies and discusses potential threats to the validity of our empirical evaluation, categorized into internal validity, external validity, construct validity, and reliability, along with the mitigation we implemented to address these risks.

- **Internal validity** concerns factors that may bias the causal relationship between the MoRe approach and the observed improvements in domain model quality. First, the F1 scores depend on $SemMatch(\cdot)$ to establish semantic matches between the generated and reference models, which further relies on embedding similarity thresholds and string-to-embedding conversion rules (e.g., $toStr(\cdot)$). Although the MiniLM-L6-V2 embedding model may fail to capture nuanced domain-specific semantics, we mitigated this issue by calibrating all thresholds in a dedicated pilot study using semantically equivalent and non-equivalent model element mutants. Second, reference models were manually constructed and reviewed by two authors (with an inter-rater agreement of 86.7%), which introduces potential subjectivity from divergent interpretations of ambiguous requirements. To reduce this threat, we: (1) unified modeling standards based on object-oriented best practices before construction; (2) resolved all discrepancies through consensus discussion; and (3) excluded or adjusted highly ambiguous requirement sentences (e.g., incomplete business rules and unbounded domain contexts).
- **External validity** refers to the generalizability of our findings to other contexts, datasets, or LLMs. First, our 30-domain problem set, collected from existing literature, covers 8 broad domains with requirements of 4–30 sentences and reference models of 3–15 classes, though niche or highly complex domains remain underrepresented. Future work will expand the dataset to address this limitation. Second, we evaluated four open-source LLMs (DeepSeek and Qwen3-Coder families) but not closed-source models. Evaluating all state-of-the-art LLMs is very challenging due to time and budget restrictions. However, the selected open-source LLMs achieve performance comparable to state-of-the-art closed-source models on many tasks, supporting their representativeness in our study.
- **Construct validity** addresses whether our chosen F1 scores accurately measure *domain model quality*. F1 score is a standard metric widely used in related work [32–34] to quantify the structural consistency and completeness of generated domain models,

making it a reasonable and comparable choice. An alternative is *LLM-as-a-judge*, but we did not adopt it due to concerns about randomness and hallucination in judge LLMs. Instead, we used our semantic model matcher to ensure stable, objective, and reproducible evaluation results. Additional metrics, such as design flexibility and real-world usability, also exist, but they typically depend on human judgment. This reliance on human evaluation brings in subjectivity and makes it challenging to perform large-scale assessments without risking researcher bias.

- **Reliability** concerns the consistency and repeatability of our experimental results. The major threat to reliability is that LLMs produce non-deterministic outputs, even with identical prompts, which could affect score variability. We mitigated this by running each experiment 5 times (per approach/LLM) and reporting average F1 scores with standard deviations (Tables 4 and 6). We also performed significance tests (Tables 5 and 7) to determine whether the observed improvements were statistically significant. In addition, the evaluation results and conclusion may be strengthened by incorporating human expert evaluation of generated models and reporting inter-annotator agreement for the reference models. However, given the scale of our study (multiple LLMs, multiple repetitions, multiple baselines, and 30 problems), a comprehensive human evaluation was not feasible within the scope of this work. The human expert evaluation will be our future work to enhance our experiment.

9. Conclusions and Future Work

In this paper, we proposed MoRe, an LLM-based approach to domain model generation from textual requirements. It employs a hybrid symbolic–LLM refinement pipeline to address the challenges of hallucination and structural inconsistency. The core idea is to employ LLMs and a rule-based checker to identify and fix common anti-patterns in domain modeling, with four major steps consisting of initial generation, model correction, model reduction and rule-based fixing. Our evaluation on 30 domain problems across four LLMs (DeepSeek V3.2, DeepSeek-R1-Distill-Llama-8B, Qwen3-Coder-480B-A35B-Instruct, Qwen3-Coder-30B-A3B-Instruct) demonstrates that MoRe improves the quality of generated domain models (measured by F1 scores via a semantic model matcher) compared to baseline approaches (Simple prompt and Iterative generation), with the sole exception of DeepSeek-R1-Distill-Llama-8B, where no statistically significant improvement was observed (see Tables 5 and 6).

For future work, we will expand MoRe from static structures in class diagrams to operations and messages. We also aim to support additional anti-patterns and explore the use of design patterns to enhance modeling. Our evaluation will be improved with more realistic modeling problems and a more diverse set of LLMs across model families and sizes. Finally, we will optimize our tool support and integrate it into modern modeling frameworks.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/electronics15061239/s1, Source code and raw data of MoRe.

Author Contributions: Conceptualization, X.H.; methodology, X.H., J.S.; software, R.C., J.S.; validation, R.C., J.S.; resources, X.H.; writing—original draft preparation, R.C.; writing—review and editing, X.H. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding. The APC was funded by the University of Science and Technology Beijing.

Data Availability Statement: The programs, benchmark problems, and experimental results presented in this study are included in the Supplementary Materials. Further inquiries can be directed at the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

NL	Natural language
NLP	Natural language processing
LLM	Large language model
MDE	Model-driven engineering
EMF	Eclipse Modeling Framework

Appendix A. Prompt Templates of MoRe

This appendix presents the prompt templates used in MoRe.

Step 1: Initial Generation

```
# Task
You are a domain modeling expert for UML Class diagram. You must carefully analyze <Description> and
create a conceptual model that represents the domain knowledge provided.

# Description
{{ system_description }}

# Output Format
```result:domain_model
@startuml
...
@enduml
```
```

Step 2.1: Name Correction

```
# Task
You are a senior domain modeler. Please append and revise the names for associations and compositions to
<Domain Model> just created previously according to <Description>.

# Description
{{ system_description }}

# Domain Model
```plantuml
{{ domain_model }}
```

Guidelines
1. Every association and composition must has a reasonable name based on its semantics.
2. Do not specify role names. Treat the association/composition name as the target role name.
3. Do not use generic names, nor use verbs, such as `for`, `has`, `contains`, `parts_of`, `applies`, and `refers_to`.

Output
```think
<Name change suggestions here>
```

```result:revision
@startuml
<your revised model in PlantUML here. One line for each change.>
@enduml
```
```

Step 2.2: Concept Correction

Task
You are a senior domain modeler. Please correct <Domain Model> previously created according to <Description>. Strictly follow the <Output> format.

Guidelines
1. Correct conceptual misuses. Check for the following modeling anti-patterns:
  a. Classes misused for enumerations or primitive types;
  b. Associations misused for inheritance relationships;
  c. Inheritance relationships misused for associations;
  d. Associations misused for message passing and invocations;
2. Adjust unreasonable inheritance hierarchy.
3. Check multiplicities and relationship direction. Treat all associations/compositions as unidirectional (`-->` and `*--`). Do not specific source end multiplicities.

Output
```think
<Model change suggestions here. One line for each change.>
```

```result:revision
@startuml
<your revised model in PlantUML here>
@enduml
```

Step 3.1: Model Reduction

Task
You are a senior domain modeler. Please review and adjust <Domain Model> according to <Description>. Strictly follow the <Output> format.

Description
{{ system_description }}

Domain Model
```plantuml
{{ domain_model }}
```

Guidelines
1. For every class, check if it denotes a domain concept that holds structural data or enables behavioral power implied in <Description>.
2. For every attribute, check if it is implied in <Description> and is NOT derived.
3. For every association/composition, check if it represents a structural pair (BUT NOT runtime invocation) that needs to be recorded.
4. Adjust/simplify overly-designed fragments.

Output
```think
<Model change suggestions here. One line for each change.>
```

```result:revision
@startuml
<your revised model in PlantUML here>
@enduml
```

Step 3.2: Grammar Correction

Note that the grammar example is omitted in the figure due to space limitation.

Task
You are a PlantUML expert. Please correct the grammar of <Domain Model> previously created. Strictly follow the <Output> format.

Guidelines
1. Correct PlantUML grammar errors.
2. Ensure that every attribute has a name and a data type (or enum), e.g., `attr : type`.
3. Ensure that every association conforms to the format `SRC_CLS --> "MULT" TAR_CLS : REF_NAME`.
4. Ensure that every composition conforms to the format `SRC_CLS *-- "MULT" TAR_CLS : REF_NAME`.
5. Ensure that inheritance e.g., `A --|> B` has no name.
6. Remove all PlantUML notes because they are not parsable.

PlantUML Example
The following example illustrates the grammar of PlantUML.

… (omitted)

Output
```result:revision
@startuml
<your revised model in PlantUML here>
@enduml
```

Step 4: Review and Fixing

Note that we only show the review request for empty classes due to space limitation.

Task
You are a domain modeling expert for UML Class diagram. To create a conceptual model that represents the domain knowledge provided, you should output revision decisions by considering <Description> and following <Guidelines> below.

Guidelines
{%- if empty_classes %}
+ **Empty Classes**: Double check every candidate of empty classes listed below to determine if it is unessential. An empty class may fall into one of the three categories: (1) an over-designed class; (2) an insignificant class; (3) an explicitly mentioned but undetailed class in <Description>. Either (1) or (2) indicates that the empty class is unessential.
{%- for c in empty_classes %}
{{ loop.index }}. `{{ c.name }}`
{%- endfor %}
{%- endif %}
… (other issues are omitted)
+ Carefully weigh your answer before giving the final judgment.

Description
{{ system_description }}

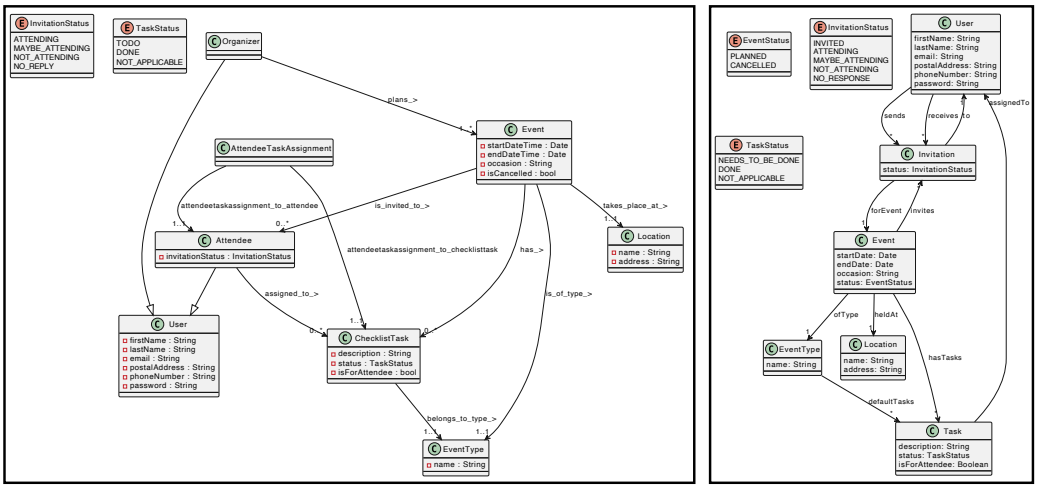
Output
```think
<Your thinking process step-by-step and actionable change suggestions here>
```

{%- if empty_classes %}
```result:empty_classes
<Output Yes or No for every empty class provided, e.g., `1. Yes -- <class_name>`. One class for each line>
```

{%- endif %}
```
… (other issues are omitted)
```

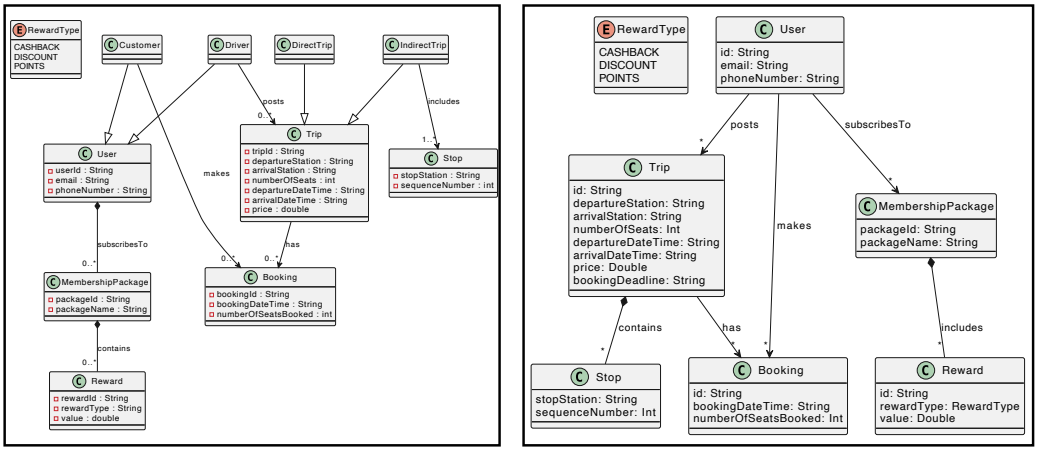
Appendix B. Example of Domain Model Generation with MoRe

Example 1: CelO Application (Deepseek V3.2)



Initial Model Final Model

Example 2: Online Rideshare System (Qwen3-Coder-480B-A35B-Instruct)



Initial Model Final Model

References

1. Naveed, H.; Khan, A.U.; Qiu, S.; Saqib, M.; Anwar, S.; Usman, M.; Akhtar, N.; Barnes, N.; Mian, A. A Comprehensive Overview of Large Language Models. *ACM Trans. Intell. Syst. Technol.* **2025**, *16*, 1–72. [CrossRef]
2. Terragni, V.; Vella, A.; Roop, P.; Blincoe, K. The Future of AI-Driven Software Engineering. *ACM Trans. Softw. Eng. Methodol.* **2025**, *34*, 1–20. [CrossRef]
3. Gao, S.; Wang, C.; Gao, C.; Lyu, M.R. SEER: Enhancing Chain-of-Thought Code Generation through Self-Exploring Deep Reasoning. *arXiv* **2025**, arXiv:2510.17130. [CrossRef]
4. Liu, Y.; Zhang, L.; Liu, F.; Wang, Z.; Wei, D.; Yang, Z.; Zhang, K.; Li, J.; Shi, L. RepoScope: Leveraging Call Chain-Aware Multi-View Context for Repository-Level Code Generation. *arXiv* **2025**, arXiv:2507.14791. [CrossRef]
5. Li, Q.; Dai, X.; Li, X.; Zhang, W.; Wang, Y.; Tang, R.; Yu, Y. CodePRM: Execution Feedback-enhanced Process Reward Model for Code Generation. In *Proceedings of the Findings of the Association for Computational Linguistics: ACL 2025*; Association for Computational Linguistics: Vienna, Austria, 2025; pp. 8169–8182. [CrossRef]
6. Zhang, Z.; Wang, C.; Wang, Y.; Shi, E.; Ma, Y.; Zhong, W.; Chen, J.; Mao, M.; Zheng, Z. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *Proc. Acm Softw. Eng.* **2025**, *2*, 481–503. [CrossRef]
7. Makharev, V.; Ivanov, V. Code Summarization Beyond Function Level. In *Proceedings of the 2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code), Ottawa, ON, Canada, 3 May 2025* ; IEEE: Piscataway, NJ, USA, 2025; pp. 153–160. [CrossRef]

8. Dhulshette, N.; Shah, S.; Kulkarni, V. Hierarchical Repository-Level Code Summarization for Business Applications Using Local LLMs. In *Proceedings of the 2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code), Ottawa, ON, Canada, 3 May 2025*; IEEE: Piscataway, NJ, USA, 2025; pp. 145–152. [CrossRef]
9. Zeng, Q.; Zhang, Y.; Ma, Z.; Jiang, B.; Sun, N.; Stol, K.; Mou, X.; Liu, H. Evaluating Generated Commit Messages with Large Language Models. *arXiv* **2025**, arXiv:2507.10906. [CrossRef]
10. Yang, G.; Zhou, Y.; Chen, X.; Zheng, W.; Hu, X.; Zhou, X.; Lo, D.; Chen, T. Code-DiTing: Automatic Evaluation of Code Generation without References or Test Cases. In *Proceedings of the 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), Seoul, Republic of Korea, 16–20 November 2025*; IEEE: Piscataway, NJ, USA, 2025; pp. 154–165. [CrossRef]
11. Navarro, J.; Ibarra, R. Automatic test case generation using natural language processing: A systematic mapping study. *Inf. Softw. Technol.* **2026**, *189*, 107929. [CrossRef]
12. Zhao, Y.; Huang, Z.; Ma, Y.; Li, R.; Zhang, K.; Jiang, H.; Liu, Q.; Zhu, L.; Su, Y. RePair: Automated Program Repair with Process-based Feedback. In *Proceedings of the Findings of the Association for Computational Linguistics: ACL 2024*; Association for Computational Linguistics: Bangkok, Thailand, 2024; pp. 16415–16429. [CrossRef]
13. Mavalankar, A.; Mansoor, H.; Marinho, Z.; Samsikova, M.; Schaul, T. AuPair: Golden Example Pairs for Code Repair. *arXiv* **2025**, arXiv:2502.18487. [CrossRef]
14. Zhang, Z.; Zhang, A.; Li, M.; Smola, A. Automatic Chain of Thought Prompting in Large Language Models. *arXiv* **2023**, arXiv:2210.03493. [CrossRef]
15. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 1877–1901. Available online: https://papers.nips.cc/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf (accessed on 12 February 2026).
16. Zhou, D.; Schärli, N.; Hou, L.; Wei, J.; Scales, N.; Wang, X.; Schuurmans, D.; Cui, C.; Bousquet, O.; Le, Q.V.; et al. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. *arXiv* **2023**, arXiv:2205.10625. [CrossRef]
17. Zhang, Y.; Du, L.; Cao, D.; Fu, Q.; Liu, Y. An Examination on the Effectiveness of Divide-and-Conquer Prompting in Large Language Models. *arXiv* **2024**, arXiv:2402.05359. [CrossRef]
18. Gao, C.; Hu, X.; Gao, S.; Xia, X.; Jin, Z. The Current Challenges of Software Engineering in the Era of Large Language Models. *ACM Trans. Softw. Eng. Methodol.* **2025**, *34*, 1–30. [CrossRef]
19. Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; et al. Self-Refine: Iterative Refinement with Self-Feedback. In *Proceedings of the Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Nice, France, 2023; Volume 36, pp. 46534–46594. Available online: https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf (accessed on 12 February 2026).
20. Ben Chaaben, M.; Burgueño, L.; David, I.; Sahraoui, H. On the Utility of Domain Modeling Assistance with Large Language Models. *arXiv* **2025**, arXiv:2410.12577. [CrossRef]
21. Iscoe, N.; Williams, G.; Arango, G. Domain modeling for software engineering. In *Proceedings of the 13th International Conference on Software Engineering, Austin, TX, USA, 13–16 May 1991*; IEEE: Piscataway, NJ, USA, 1991; pp. 340–343. [CrossRef]
22. Prieto-Diaz, R.; Arango, G. *Domain Analysis and Software Systems Modeling*; IEEE Computer Society Press: Washington, DC, USA, 1991.
23. Górski, T. Software architecture description in original software publications. *Softw. Impacts* **2026**, *27*, 100802. [CrossRef]
24. Yue, T.; Briand, L.C.; Labiche, Y. A systematic review of transformation approaches between user requirements and analysis models. *Requir. Eng.* **2011**, *16*, 75–99. [CrossRef]
25. Bozyigit, F.; Bardakci, T.; Khalilipour, A.; Challenger, M.; Ramackers, G.; Babur, Ö.; Chaudron, M.R.V. Generating domain models from natural language text using NLP: A benchmark dataset and experimental comparison of tools. *Softw. Syst. Model.* **2024**, *23*, 1493–1511. [CrossRef]
26. Yue, T.; Briand, L.C.; Labiche, Y. Facilitating the transition from use case models to analysis models: Approach and experiments. *ACM Trans. Softw. Eng. Methodol.* **2013**, *22*, 1–38. [CrossRef]
27. Thakur, J.S.; Gupta, A. Identifying domain elements from textual specifications. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, Singapore, 3–7 September 2016*; ASE '16; Association for Computing Machinery: New York, NY, USA, 2016; pp. 566–577. [CrossRef]
28. Lucassen, G.; Robeer, M.; Dalpiaz, F.; van der Werf, J.M.E.M.; Brinkkemper, S. Extracting conceptual models from user stories with Visual Narrator. *Requir. Eng.* **2017**, *22*, 339–358. [CrossRef]
29. Arora, C.; Sabetzadeh, M.; Briand, L.; Zimmer, F. Extracting domain models from natural-language requirements: Approach and industrial evaluation. In *Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems, Saint-Malo, France, 2–7 October 2016*; MODELS '16; Association for Computing Machinery: New York, NY, USA, 2016; pp. 250–260. [CrossRef]
30. Cámara, J.; Troya, J.; Burgueño, L.; Vallecillo, A. On the assessment of generative AI in modeling tasks: An experience report with ChatGPT and UML. *Softw. Syst. Model.* **2023**, *22*, 781–793. [CrossRef]

31. Chen, K.; Yang, Y.; Chen, B.; López, J.A.H.; Mussbacher, G.; Varró, D. Automated Domain Modeling with Large Language Models: A Comparative Study. In *Proceedings of the 2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS), Västerås, Sweden, 1–6 October 2023*; IEEE: Piscataway, NJ, USA, 2023; pp. 162–172. [CrossRef]
32. Yang, Y.; Chen, B.; Chen, K.; Mussbacher, G.; Varró, D. Multi-step Iterative Automated Domain Modeling with Large Language Models. In *Proceedings of the Proceedings: MODELS 2024—ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, Linz, Austria, 22–27 September 2024*; Association for Computing Machinery, Inc.: New York, NY, USA, 2024; pp. 587–595. [CrossRef]
33. Silva, J.; Ma, Q.; Cabot, J.; Kelsen, P.; Proper, H.A. Application of the Tree-of-Thoughts Framework to LLM-Enabled Domain Modeling. In *Proceedings of the Conceptual Modeling*; Springer: Cham, Switzerland, 2025; pp. 94–111. [CrossRef]
34. Wang, Y.; Ge, N.; Liu, J.; Cao, Z.; Chen, Z.; Hu, C. Generating SysML Behavior Models via Large Language Models: An Empirical Study. In *Proceedings of the 16th International Conference on Internetware, Trondheim, Norway, 20–22 June 2025*; Internetware '25; Association for Computing Machinery: New York, NY, USA, 2025; pp. 366–377. [CrossRef]
35. Apvrille, L.; Sultan, B. System Architects Are not Alone Anymore: Automatic System Modeling with AI. In *MODELSWARD 2024: 12th International Conference on Model-Based Software and Systems Engineering*; SCITEPRESS-Science and Technology Publications: Setúbal, Portugal, 2024; pp. 27–38. [CrossRef]
36. Wang, W.; Wei, F.; Dong, L.; Bao, H.; Yang, N.; Zhou, M. MINILM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 5776–5788. [CrossRef]
37. Saini, R.; Mussbacher, G.; Guo, J.L.; Kienzle, J. Machine learning-based incremental learning in interactive domain modelling. In *Proceedings of the 25th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS 2022*; Association for Computing Machinery, Inc.: New York, NY, USA, 2022; pp. 176–186. [CrossRef]
38. Bolloju, N.; Leung, F.S. Assisting novice analysts in developing quality conceptual models with UML. *Commun. ACM* **2006**, *49*, 108–112. [CrossRef]
39. Gavrilova, T.; Onufriev, V. Conceptual Modelling: Common Students' Mistakes in Visual Representation. In *Proceedings of the Teaching and Learning in a Digital World*; Springer: Cham, Switzerland, 2018; pp. 199–209. [CrossRef]
40. Chourio, P.; Azevedo, R.; Castro, A.; Gadelha, B. Most common errors in software modeling using UML. In *Proceedings of the XXXIII Brazilian Symposium on Software Engineering, Salvador, Brazil, 23–27 September 2019*; SBES '19; Association for Computing Machinery: New York, NY, USA, 2019; pp. 244–253. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.